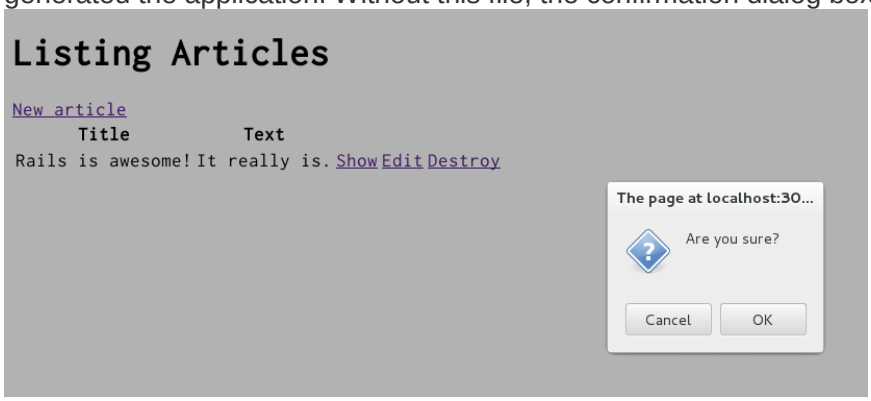
```
      <td><%= link_to 'Edit', edit_article_path(article) %></td>
      <td><%= link_to 'Destroy', article_path(article),
              method: :delete,
              data: { confirm: 'Are you sure?' } %></td>
    </tr>
  <% end %>
</table>
```

Here we're using `link_to` in a different way. We pass the named route as the second argument, and then the options as another argument. The `:method` and `:'data-confirm'` options are used as HTML5 attributes so that when the link is clicked, Rails will first show a confirm dialog to the user, and then submit the link with method `delete`. This is done via the JavaScript file `jquery_ujs` which is automatically included into your application's layout (`app/views/layouts/application.html.erb`) when you generated the application. Without this file, the confirmation dialog box wouldn't appear.



Learn more about jQuery Unobtrusive Adapter (jQuery UJS) on Working With Javascript in Rails guide.

Congratulations, you can now create, show, list, update and destroy articles.

In general, Rails encourages using resources objects instead of declaring routes manually. For more information about routing, see Rails Routing from the Outside In.

6 Adding a Second Model

It's time to add a second model to the application. The second model will handle comments on articles.

6.1 Generating a Model

We're going to see the same generator that we used before when creating the `Article` model. This time we'll create a `Comment` model to hold reference of article comments. Run this command in your terminal:

```
$ bin/rails generate model Comment commenter:string body:text
article:references
```

This command will generate four files:

File	Purpose
db/migrate/20140120201010_create_comments.rb	Migration to create the comments table in your database (your name will include a different timestamp)
app/models/comment.rb	The Comment model
test/models/comment_test.rb	Testing harness for the comments model